

Credal Cluster Tree Elimination: Computing All Marginals via Two-Pass Bucket Tree Propagation

IBM Research

Abstract

Credal Cluster Tree Elimination (CTE) extends bucket elimination to compute lower and upper marginal probability bounds for *all* variables in a credal network within a single two-pass execution. The first pass (collect) is identical to standard credal variable elimination, sending messages from leaves to root. The second pass (distribute) propagates global information from root back to leaves. After both passes, every bucket has sufficient information to extract its local marginal bounds. The algorithm supports epsilon-approximate pruning at all stages, yielding an FPTAS for all-marginals inference.

1 Background: From Single-Query to All-Marginals

1.1 Bucket Elimination (Single Query)

Standard credal variable elimination (CVE), as described in the companion document `credal_ve.tex`, computes bounds for a *single query variable* Z . Given an elimination ordering, it processes buckets from first to last. The final bucket produces the marginal for Z . To query a different variable, the entire elimination must be re-run with a different ordering.

1.2 Bucket Tree Elimination (All Queries)

Bucket tree elimination organizes the buckets into a **tree** and performs **two passes**:

1. **Collect (upward)**: identical to standard VE — messages flow from leaves to root.
2. **Distribute (downward)**: messages flow from root back to leaves, propagating global information.

After both passes, every bucket has access to the full joint information, enabling extraction of all marginals simultaneously.

This is equivalent to the junction tree (cluster tree) algorithm of Lauritzen and Spiegelhalter (1988), viewed through the bucket elimination framework of Dechter (1999).

2 Potentials (Recap)

A **potential** $\phi(\mathbf{Y})$ is a finite set of non-negative real-valued functions over the joint domain of variables $\mathbf{Y}$. The three operations are:

- **Combine**: $\phi \cdot \psi = \{f \cdot g : f \in \phi, g \in \psi\}$
- **Marginalize**: $\sum_X \phi = \{\sum_x f : f \in \phi\}$
- **Prune**: remove dominated functions (exact or ε -approximate)

3 Algorithm

3.1 Step 1: Bucket Tree Construction

Given an elimination ordering $\mathbf{o} = (X_1, X_2, \dots, X_n)$ and a set of initial potentials (one per credal factor plus evidence indicators):

1. **Assign potentials to buckets.** Each potential is assigned to the bucket of the first variable in $\mathbf{o}$ that appears in the potential's scope.
2. **Determine parent-child relationships.** For each bucket X_i , compute the *message scope*: the union of scopes of all potentials in the bucket (including received messages), minus X_i itself. The **parent** of X_i is the next variable in $\mathbf{o}$ that appears in this message scope. The message scope is propagated to the parent's effective scope.

The result is a rooted tree where the last variable in the ordering is the root.

3.2 Step 2: Collect Pass (Upward)

Process buckets in elimination order (leaves to root). At each bucket X_i :

1. **Gather:** local potentials + upward messages λ_{X_j} from all children X_j .
2. **Combine:** $\psi_i = \prod \{\text{gathered potentials}\}$.
3. **Marginalize:** $\lambda_{X_i} = \sum_{X_i} \psi_i$.
4. **Prune:** $\lambda_{X_i} \leftarrow \text{prune}(\lambda_{X_i})$.
5. **Send** λ_{X_i} to the parent bucket.

This is identical to standard CVE. After the collect pass, the root bucket has “seen” all information.

3.3 Step 3: Distribute Pass (Downward)

Process buckets in *reverse* elimination order (root to leaves). At each bucket X_i , for each child X_j :

1. **Gather:** the downward message π_{X_i} from X_i 's own parent + local potentials in X_i 's bucket + upward messages from all children of X_i *except* X_j .
2. **Combine:** $\psi = \prod \{\text{gathered potentials}\}$.
3. **Marginalize:** $\pi_{X_j} = \sum_{X_i} \psi$ (sum out X_i).
4. **Prune:** $\pi_{X_j} \leftarrow \text{prune}(\pi_{X_j})$.
5. **Send** π_{X_j} to child X_j .

After the distribute pass, every bucket has received information from the rest of the network via its parent's downward message.

3.4 Step 4: Extract All Marginals

At each bucket X_i , the full information is:

$$\Phi_i = \underbrace{\text{local potentials}}_{\text{assigned in Step 1}} \cdot \underbrace{\prod_{j \in \text{children}(i)} \lambda_{X_j}}_{\text{from collect}} \cdot \underbrace{\pi_{X_i}}_{\text{from distribute}}.$$

Marginalize out all variables except X_i , then normalize each function and take componentwise min/max for the lower/upper bounds.

3.5 Pseudocode

Algorithm 1 Credal Cluster Tree Elimination (CTE)

Require: Extreme points, evidence $\mathbf{Y} = \mathbf{y}$, elimination heuristic, optional ε

Ensure: Bounds $[\underline{P}(X_i=x), \overline{P}(X_i=x)]$ for **all** variables X_i and states x

- 1: Build potentials from extreme points and evidence indicators
- 2: Compute elimination ordering $\mathbf{o}$
- 3: Build bucket tree: assign potentials, determine parent-child

▷ COLLECT (UPWARD)

- 4: **for** each X_i in $\mathbf{o}$ (leaves to root) **do**
- 5: $\psi_i \leftarrow \prod \{\text{local pots} \cup \{\lambda_{X_j} : j \in \text{ch}(i)\}\}$
- 6: $\lambda_{X_i} \leftarrow \text{prune}(\sum_{X_i} \psi_i)$
- 7: Send λ_{X_i} to parent
- 8: **end for**

▷ DISTRIBUTE (DOWNWARD)

- 9: **for** each X_i in reverse $\mathbf{o}$ (root to leaves) **do**
- 10: **for** each child X_j of X_i **do**
- 11: $\psi \leftarrow \prod \{\pi_{X_i}, \text{local pots}, \{\lambda_{X_k} : k \in \text{ch}(i) \setminus \{j\}\}\}$
- 12: $\pi_{X_j} \leftarrow \text{prune}(\sum_{X_i} \psi)$
- 13: Send π_{X_j} to child X_j
- 14: **end for**
- 15: **end for**

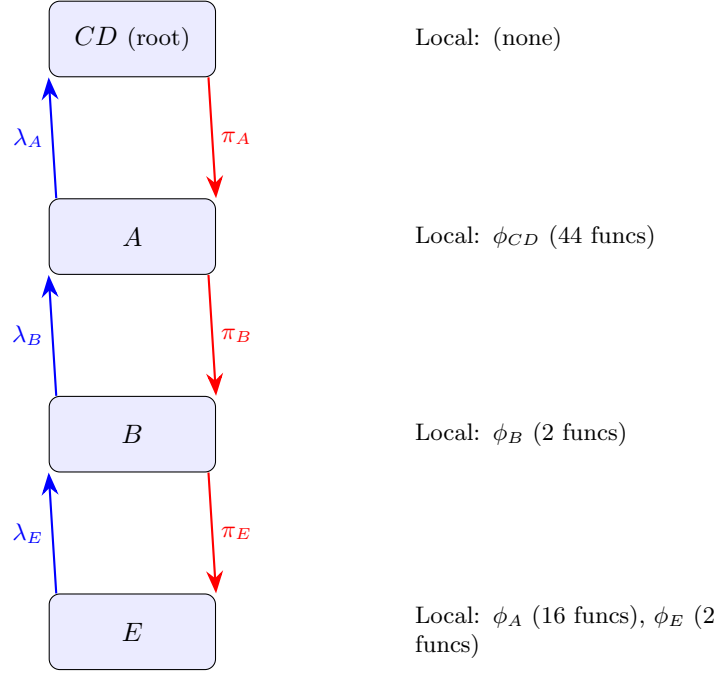
▷ EXTRACT MARGINALS

- 16: **for** each variable X_i **do**
 - 17: $\Phi_i \leftarrow \prod \{\text{local pots}, \{\lambda_{X_j} : j \in \text{ch}(i)\}, \pi_{X_i}\}$
 - 18: $\phi_i \leftarrow \sum_{\mathbf{Y} \setminus \{X_i\}} \Phi_i$
 - 19: $\underline{P}(X_i=x) \leftarrow \min_{f \in \phi_i} f(x) / \sum_{x'} f(x')$
 - 20: $\overline{P}(X_i=x) \leftarrow \max_{f \in \phi_i} f(x) / \sum_{x'} f(x')$
 - 21: **end for**
-

4 Detailed Example: Alarm Network

We trace CTE on the alarm network (`alarm.lcn`) with variables E, B, A, CD and elimination ordering $\mathbf{o} = (E, B, A, CD)$.

4.1 Bucket Tree



The bucket tree structure:

Bucket	Parent	Children	Local Potentials	Funcs
E	B	—	ϕ_A : scope $\{A, B, E\}$; ϕ_E : scope $\{E\}$	16, 2
B	A	$\{E\}$	ϕ_B : scope $\{B\}$	2
A	CD	$\{B\}$	ϕ_{CD} : scope $\{CD, A\}$	44
CD	— (root)	$\{A\}$	(none)	0

The factor ϕ_A (scope $\{A, B, E\}$) is assigned to bucket E because E is the first variable in the ordering that appears in ϕ_A 's scope. Similarly, ϕ_{CD} (scope $\{CD, A\}$) goes to bucket A .

4.2 Collect Pass (Upward Messages)

Bucket E : Combine ϕ_A (16 funcs) $\times$ ϕ_E (2 funcs) = 32 functions over $\{A, B, E\}$. Marginalize out E :

$$\lambda_E(\cdot) = \sum_e \psi_E(A, B, e), \quad \text{scope: } \{A, B\}, \quad 32 \text{ functions.}$$

After pruning: 32 functions remain. Each function $f_k(A, B)$ is a 2×2 array, e.g.:

$$f_0 = \begin{pmatrix} 0.860 & 0.000 \\ 0.140 & 1.000 \end{pmatrix}, \quad f_1 = \begin{pmatrix} 0.908 & 0.000 \\ 0.092 & 1.000 \end{pmatrix}, \quad \dots$$

(rows = $A \in \{0, 1\}$, columns = $B \in \{0, 1\}$).

λ_E is sent to parent bucket B .

Bucket B : Combine ϕ_B (2 funcs) $\times$ λ_E (32 funcs) = 64 functions over $\{A, B\}$. Marginalize out B :

$$\lambda_B(A) = \sum_b \psi_B(A, b), \quad \text{scope: } \{A\}, \quad 59 \text{ functions (after pruning)}.$$

Example functions: $f_0 = [0.688, 0.312]$, $f_1 = [0.726, 0.274]$, $\dots$

λ_B is sent to parent bucket A .

Bucket A : Combine ϕ_{CD} (44 funcs) $\times$ λ_B (59 funcs) = 2596 functions over $\{CD, A\}$. Marginalize out A :

$$\lambda_A(CD) = \sum_a \psi_A(CD, a), \quad \text{scope: } \{CD\}, \quad 2584 \text{ functions (after pruning)}.$$

λ_A is sent to root bucket CD .

Bucket CD (root): No local potentials. Combines λ_A and marginalizes CD : λ_{CD} is a scalar (scope $\emptyset$), 352 functions. This is the normalization constant.

4.3 Distribute Pass (Downward Messages)

Root $CD \rightarrow A$: The root has no parent message and no local potentials. The downward message to A is trivial:

$$\pi_A = \text{scalar potential with 1 function: } f_0 = [4.0].$$

This scalar acts as a normalization factor.

$A \rightarrow B$: Combine π_A (1 func) $\times$ ϕ_{CD} (44 funcs, local to A) = 44 functions over $\{CD, A\}$. Marginalize out A :

$$\pi_B(CD) = \sum_a \psi(CD, a), \quad \text{scope: } \{CD\}, \quad 44 \text{ functions}.$$

π_B carries information from the CD factor “above” B in the tree.

$B \rightarrow E$: Combine π_B (44 funcs) $\times$ ϕ_B (2 funcs, local to B) = 88 functions. Marginalize out B :

$$\pi_E(CD) = \sum_b \psi(CD, b), \quad \text{scope: } \{CD\}, \quad 60 \text{ functions}.$$

π_E tells bucket E what the rest of the network (above E) “thinks” about CD .

4.4 Marginal Extraction

At each bucket, combine all available information:

Bucket E : $\Phi_E = \phi_A \cdot \phi_E \cdot \pi_E$. Marginalize out A, B, CD to get $\phi(E)$. Normalize each function, take min/max:

$$P(E=0) \in [0.9000, 0.9500], \quad P(E=1) \in [0.0500, 0.1000].$$

Bucket B : $\Phi_B = \phi_B \cdot \lambda_E \cdot \pi_B$. Marginalize out A, CD :

$$P(B=0) \in [0.8000, 0.9000], \quad P(B=1) \in [0.1000, 0.2000].$$

Bucket A : $\Phi_A = \phi_{CD} \cdot \lambda_B \cdot \pi_A$. Marginalize out CD :

$$P(A=0) \in [0.6880, 1.0000], \quad P(A=1) \in [0.0000, 0.3120].$$

Bucket CD (root): $\Phi_{CD} = \lambda_A$ (only child message, no parent). Already over $\{CD\}$:

$$P(CD=k) \in [0.0, 1.0] \text{ for all } k \in \{0, 1, 2, 3\}.$$

The wide bounds for CD reflect the large number of extreme points (11 for $A=1$) and the vacuous prior.

4.5 Summary of Messages

Message	Direction	Scope	# Functions
λ_E	$E \rightarrow B$ (up)	$\{A, B\}$	32
λ_B	$B \rightarrow A$ (up)	$\{A\}$	59
λ_A	$A \rightarrow CD$ (up)	$\{CD\}$	2584
λ_{CD}	root (up)	$\emptyset$	352
π_A	$CD \rightarrow A$ (down)	$\emptyset$	1
π_B	$A \rightarrow B$ (down)	$\{CD\}$	44
π_E	$B \rightarrow E$ (down)	$\{CD\}$	60

5 Epsilon Approximation

The ε -approximate version replaces exact pruning with ε -pruning at **all three stages**:

1. After each upward message is marginalized (collect pass).
2. After each downward message is marginalized (distribute pass).
3. After combining all information at each bucket (marginal extraction).

Definition 1 (ε -Pruning in CTE). *A function f is removed from a potential if there exists another function g such that $g(\mathbf{y}) \geq f(\mathbf{y}) - \varepsilon$ for all $\mathbf{y}$.*

Since ε -pruning is applied in both passes, the effective error at any bucket is bounded by $O(\varepsilon \times d)$ where d is the bucket's depth in the tree. For the alarm network:

	$P(E=1)$	$P(B=1)$	$P(A=1)$	Time (s)
Exact ($\varepsilon=0$)	[0.05, 0.10]	[0.10, 0.20]	[0.00, 0.31]	50.0
$\varepsilon=0.01$	[0.05, 0.10]	[0.10, 0.20]	[0.00, 0.31]	11.7

The ε -approximate version is $4\times$ faster with identical bounds on this small network. On larger networks, the speedup from controlling potential cardinality growth is more dramatic.

Theorem 1 (FPTAS for All Marginals). *For credal networks of bounded treewidth w^* and bounded variable cardinality k , the ε -approximate CTE runs in time polynomial in the number of variables n , $1/\varepsilon$, and k^{w^*} , and produces bounds within $O(n\varepsilon)$ of the exact bounds for all variables simultaneously.*

6 Comparison with Single-Query VE

Property	CVE (single query)	CTE (all marginals)
Passes	1 (upward only)	2 (upward + downward)
Queries per run	1	All n variables
Total work for n queries	$O(n)$ runs	1 run
Message storage	Not needed	Store all λ, π
Space	$O(C \cdot k^{w^*})$	$O(n \cdot C \cdot k^{w^*})$

CTE is preferred when bounds on multiple or all variables are needed. CVE is preferred for a single query, as it avoids the overhead of the distribute pass and message storage.

7 References

1. R. Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113:41–85, 1999.
2. K. Kask, R. Dechter, J. Larrosa, F.G. Cozman. Bucket-tree elimination for automated reasoning. Technical report, UCI, 2001.
3. S.L. Lauritzen, D.J. Spiegelhalter. Local computations with probabilities on graphical structures. *JRSS-B*, 50(2):157–224, 1988.
4. D.D. Mauá, F.G. Cozman. Thirty years of credal networks. *IJAR*, 126:133–157, 2020.
5. D.D. Mauá, C.P. de Campos, M. Zaffalon. Solving limited memory influence diagrams. *JAIR*, 44:97–140, 2012.